Citation Grounded Indian Legal Research Assistant

Team ID 12

Sanskar Awasthi (S24BCAU0138)
Rishant Ranjan (S24BCAU0036)

Department of Computer Science Engineering & Technology
Bennett University, India

Abstract **The large language models are currently primarily used for the legal researches, although mostly they make up a new case name, the citations mentioned in them and section numbers which are not existing. The problem is becoming more and more relevant to Indian law than to any other country, since the users continue referring to the Indian Penal Code despite the Bharatiya Nyaya Sanhita(BNS) has replaced it in July 2024. This paper presents Nyaya which is a RAG system answering the queries on Indian law, Constitution articles and the Supreme Court judgments with an verifiable source. Firstly, the documents are divided into legal (the sections, the articles, numbered paragraphs) and the rest, with embedding of MiniLM sentence encoder. Upon asking a question, the rule-based classifier assigns weights to the deep searches path and a exact match over all other legal identifiers, after which two of the ranked lists are fused with Reciprocal Rank Fusion, and the authority is assigned before to the re-ranking of all the candidates. Next, the IPC to BNS table is included as the citable and refrencable source, and every citation is verified and summarized in the confidence score. Even though the index holds just about 112,555 chunks from the 5 of the public Hugging Face dataset that we download and use and thus is checked on 50 hand built questions with it. Hybrid retrieval gives keyword grounded precision at 4 passages from the 0.656 (dense only) to 0.811 and then the Hit from a 0.796 to 0.878. Thus every end, with a Nemotron model served by a NVIDIA NIM, our system reach a citation validity of 0.70 for now, and thus the response rate of just 0.68 which is decent one and the pattern detected hallucination rate of 0.04 only now now which is also good compared to others.**

I. INTRODUCTION

Legal research is mainly locating a , constitutional article or one that answers a question and reading it much more accurately. In majority India of this is the majority work. The Supreme Court of India and 25 High Courts produces thousands of the many ones judgments every year, central and state legislation runs upto 1000s of Acts, and criminal laws itself was written again in 2023, the Bharatiya Nyaya Sanhita (BNS), Bharatiya Nagarik Suraksha Sanhita (BNSS) and Bharatiya Sakshya Adhiniyam replaced IPC, the Code of Criminal Procedure and the Evidence Act from 1 July 2024 [21]. Lawyers , students and citizens in general now must move between 2 numbering schemes, like murder is Section 302 of the IPC but Section 103 of the BNS in it.

LLMs ussually appear to solve this problem because they answers like they all know answer, but they are'nt mcuh trustable sources of law. Dahl et al.measured that hallucination rates between 69% and 88% when popular LLM were asked with a verifiable questions about US federal cases, and Magesh et al. founded that a commercial legal research AI built using a retrieval still halucinated within the 17% to 33% range of their benchmarks query. Thus because of a wrong section number has real consequences, a useful assistant has to be shown where ever every statement comes from that sentence.

Whereas RAG addresses part of this problem by running a model on multiple paragraohs that it got from a mcuh more trusted dataset. Though much more standard of RAG pipelines, however, rely on a singler much more dense retriever.Though dense retrievers capture meaning well but handle exact once poorly a query about "Section 498A" or "Article 21" must match all identifiers fully , and the embeddings model trained on a general text just does not take away Article 21 from Article 210 or Section 302 to Section 304 . Just these Existing systems also rarely check the other citations in generated answers pointed to all the passages that were given to it , and they do not handle the IPC to BNS transition that is mainly used in Indian criminal law questions today also produces corre ct answers.

So our project introduces Nyaya a RAG AI for Indian law that treats LLMs purely as a writer over retrieved evidence. system combines chunking with hybrid search using the weighted Reciprocal Rank Fusion and a authority based re ranking, just to ensure accuracy, it includes an IPC to BNS mapping table also as an verifiable source.

The contributions of this work are

1. The structure aware chunking scheme which splits rules per sections, Constitution per article and judgments in numbered paragraphs, producing a 112,555 chunk index from 5 public Indian law datasets gives it.

2. A query adaptive hybrid retriever which also gives the each query , runs dense and exact match searches on them , and fuses 2 lists with theclasses specific RRF weights. Thus executing all of this It improve the keyword grounded precision at 4 passages by 15.5 percentage points over dense only retrieval.

3. A bidirectional of IPC/BNS's similars module (63 sections) which usually expands the retrieval query and is presented to the model as a citable source.

4. A answer verification and evaluation layer every `[SOURCE_N]` tag is validated, a 4 signals

confidence score is reported with each answer always , and six metrics (citation validity, cited claim coverage, citation support, hallucination rate, response rate and retrieval precision) that are measured on a 50 query test set, together with an ablation study and a systematic parameter sweep.

II. Literature Survey

Retrieval augmented generation. Lewis etal.[1] combined a pre trained generator with a very big index of Wikipedia that we got with Dense Passage Retrieval and trained the two of them together fully, RAG now cause of this now set new results on open field questioning answering benchmarks such as the Natural Questions and produced more of real anfd factual text than a llm with parameter based model only. Its limitations for the legal field is that the retriever is purely dense and that the model is not required to attribute statements to specific passages. Izacard and Grave [11] showed us that with Fusion in Decoder that answer quality keeps improving as more passages are read, while Liu et al. [12] already showed that LLMs use information at the start and end of a long context much better than information in the middle of whatever we give it , which argues for a small number of high quality passages.

Dense and lexical retrieval. Karpukhin et al. [7] trained the dual BERT encoder on questionpassage pairs and beat the BM25 performance by a large margin on top 20 retrieval accuracy on several of the qa datasets; they also reported that all of the writing techniques have become increasingly good at dealing with different kinds of queries, rather than on rare exact tokens, which is precisely the case for the sections and then article numbers. Reimers and Gurevych [8] proposed Sentence BERT, which is siamese architecture providing sentence embeddings for cosine similarity search, and Wang et al. [9] distilled such models into very compact MiniLM models family, which is used in this project for embedding in the chromadb in our project. Cormack et al. [2] employed Reciprocal Rank Fusion, which is unsupervised technique combining ranked lists based solely on ranks and beats more complicated fusion and learning-to-rank techniques on TREC data. The RRF technique utilizes fixed weights, however.

Models for legal fields and Indian legal corpora. Chalkidis et al. [6] pre-trained LEGAL BERT on English legislation, contract, and case law, showing improved results on legal classification tasks as compared to the general BERT model for mostly European and US legislation. Paul et al. [5] pre-trained InLegalBERT and InCaseLawBERT on Indian court judgments and showed improved results on Indian statute, semantic segmentation of Indian legislation, and prediction of Indian judgments. Malik et al. [13] created the ILDC corpus, containing approximately 35,000 Indian Supreme Court cases that are used in prediction of Indian judgments with explanation, while Kalamkar et al. [14] annotated roles in Indian judgments. The FIRE AILA track [15] benchmarked the retrieval of Indian legal questions. All these works have prepared the models and datasets, but did not create the question answering system for citing and passed the BNS.

Legal hallucinations and attribution. According to Dahl et al. [4], the legal hallucinations of GPT type model like the gpt or the llama model is based on the federal court cases, and found that models many times actually accepted the bad things and make lie holdings. Also, according to Magesh et al. [3], they tested it on commercial RAG legal tools on more than 200 questions and found that the retrieval actually reduced but does not remove hallucinations fully, many times because the retrieved document was almost always wrong. Gao et al [16] introduced the ALCE benchmark for agents which now answers with citations and found that even the biggest and strongest models almost many times produced citations that not fully support their statement. Then Ji et al. [17] surveyed the hallucination in natural language generation and identified source output divergence as the central cause of all of this.

RAG system evaluation. Es et al introduces and suggest using RAGAS, which scores faithfulness, answer relevance and context relevance with LLM judges, and Guha et al. [19] actually built LegalBench, which was on a collection of 162 legal reasoning tasks. Both are mainly made for the US law and needs either of them.

Research gap. The existing research on the topic has created 4 research gaps within the Indian legal domain (i) Firstly, standard RAG relies heavily on dense retrieval and is poor at identifying exact queries required in legal queries (ii) Secondly, fusion approaches such as RRF approach uses fixed weights regardless of whether the user requests for a section or case or concepts (iii) Thirdly, currently available systems do not verify at all who created the citations to the retrieved message, nor does it provide any signal on how good it is grounded the answer is and (iv) Fourthly no available system deals with the IPC to BNS transition. Nyaya resolves these issues with the help of query hybrid retrieval, re-ranking with authority awareness, and explicit citation validation using deterministic metrics requiring no LLM judge.

III. Proposed Methodology

A. System Architecture

It has the stages of offline index and online query (Fig. 1). In offline, the raw document is downloaded, preprocessed, segmented into structure-aware chunks, and then indexed in a persistent ChromaDB collection using cosine distance. In online, the query goes through IPC/BNS lookup, query classification, hybrid search, fusion and re-ranking, prompt construction, generation, citation verification, and confidence score. The codebase has around 3,600 lines of Python organized into ingestion, chunking, embeddings, retrieval, generation, confidence, and evaluation modules, with 250 unit and integration tests.

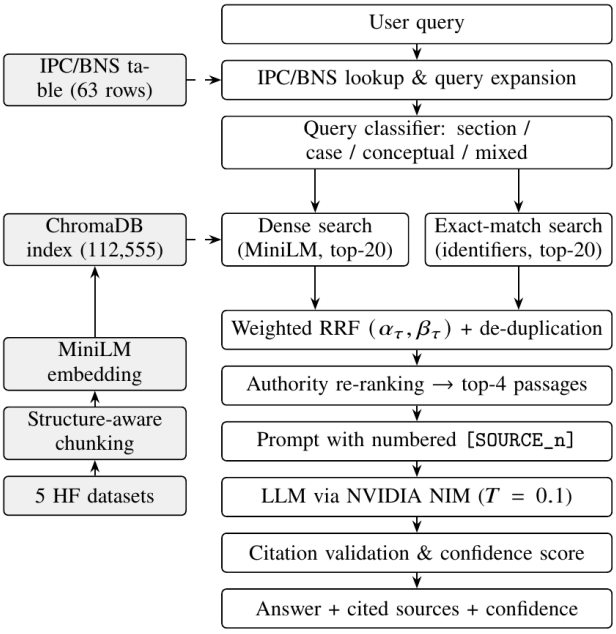


Fig. 1. Nyaya architecture. Grey boxes are data and the offline indexing stage (bottom left, read upwards) white boxes form the online query pipeline. Matching IPC/BNS rows are also appended to the prompt as a source.

B. Input Data and Preprocessing

The dataset of ours now joins five Hugging Face datasets and question answer that is then paired with Indian statutes, Supreme Court judgments with summaries, pre chunked Supreme Court judgments, a small legal RAG collection and the articles of the Constitution togeter that we had then a conversion script then finds each of the record with a document type (*statute*, *constitution* or *judgement*), also text and metadata such as court, date, case name, act name and section number. Optional scripts also add the ful text of the IPC, BNS, CrPC and also the Constitution from India Code and landmark judgments from Indian Kanoon scanned PDFs we got from the web that are then parsed with PyMuPDF and an OCR fallback using Tesseract OCR . A text cleaner then removes then page headers, footers and URL lines from the whole document , and also repairs the common OCR artefacts, which are normalisesed whitespace and standardises section references and also a regular expression metadata extractor recovers the case names, AIR/SCC citations, judges and section references.

C. Structure Aware Chunking

The Fixed size chunking cuts in half of them and separates explanations and provisos from the section they are from . Nyaya instead now chunks by document type. Statutes are split into one section per chunk, and then a section longer than the maximum is split again and again at its markers . The Constitution is then split into one article per chunk, keeping explanations and provisos with their article. Judgments are then split at the numbered paragraphs and short paragraphs are merged this is done so that ther meanings doesnt change when chunked , segments are then packed to a target length, the case name is then linked to every chunk so that every passage is known by self only .

$$\hat{T}(c) = \lfloor 1.3\,|W(c)| \rfloor, \qquad (1)$$

where $|W(c)|$ is also the number of the whitespace separated words in chunk c whic is The target, now the maximum and overlap lengths are 450, 800 and 50 which is estimated tokens. Chunk identifiers are also made from the document identifier and chunk index (`doc_idindex`), so re indexing changes the chunks in place instead of making duplicartes of it

D. Embedding and Indexing

Each chunk is embedded with the `all MiniLM L6 v2` Sentence Transformer, which maps the text to a 384 dimensional vector. Embeddings are done in batches of 64 and stored in ChromaDB with an HNSW index in cosine space. The semantic similarity of query q and chunk c is

$$s(q,c) = 1 - d_{\cos}(q,c) = \frac{\mathbf{e}_q \cdot \mathbf{e}_c}{\|\mathbf{e}_q\|\,\|\mathbf{e}_c\|}, \qquad (2)$$

where $\mathbf{e}_q$ and $\mathbf{e}_c$ are the query and chunk embeddings and $d_{\cos}$ is the cosine distance returned by the index.

E. IPC/BNS Lookup

A curated table then maps the 63 commonly used IPC sections to their BNS parts, including the sub sections (for example IP 376 to BNS 64 and IPC 420 to BNS 318). The Regular expressions now also detect IPC and BNS references in either order cause of this change (“Section 302 IPC”, “IPC 302”, “s. 302 of Indian Penal Code”) and offence names for questions.The Matched rows also expand retrieval query with counterpart section and are also formatted as extra source passage, so the model can cite the represets itself rather than recalling it from memory eachtime.

F. Query Classification and Hybrid Retrieval

A rule based classifier assigns each query a type τ. Patterns for “Section n”, Article n”, Act names and the abbreviations IPC, CrPC, CPC and BNS indicate a *section lookup* AIR/SCC citations or a case name of the form “X v Y” indicate a *case lookup* both indicate *mixed* neither indicates *conceptual*.

The *dense path* returns the 20 nearest chunks to the query embedding. The *exact match path that* extracts legal identifiers from the query section and article numbers are now normalised to a canonical form (“section 304a” becomes “Section 304A”), AIR and SCC citations, and the distinctive party names of a named case. Generic parties such as “Union of India” or “State of …” are skipped because they occur in thousands of judgments, and initials are also tried in dotted form (“MC Mehta” and “M.C. Mehta”). For each identifier, ChromaDB returns up to 100 chunks that contain it also, ordered by similarity to the query chunks where the identifier only occurs as a prefix of a longer one (“Article 21” inside “Article 210”) are discarded with a word boundary check, and the 20 nearest remain.

The two ranked lists are fused with weighted RRF [2]. For list L and chunk c at rank $r_L(c)$,

$$\mathrm{RRF}_L(c) = \frac{1}{k + r_L(c)}, \tag{3}$$

with $\mathrm{RRF}_L(c) = 0$ if $c \notin L$ and smoothing constant $k = 60$. The fused score is

$$R(c) = \alpha_\tau \,\mathrm{RRF}_{\mathrm{sem}}(c) + \beta_\tau \,\mathrm{RRF}_{\mathrm{kw}}(c),$$

where $(\alpha_\tau, \beta_\tau)$ is (0.3,0.7) for section lookups, (0.2,0.8) for case lookups, (0.8,0.2) for conceptual queries and (0.5,0.5) is for then mixed queries. Because some source datasets contain the same record several times, candidates with identical normalised text are removed after fusion now.

G. Authority Aware Re ranking

Legal sources differ in authority statute text binds, a Supreme Court judgment binds all lower courts, and a tribunal order is persuasive at most. The authority score of chunk c is $A(c) = \begin{cases} 1, & c \in \mathcal{L}, \\ 0.4\, w_{\mathrm{court}} + 0.3\, w_{\mathrm{rec}} + 0.3\, w_{\mathrm{cit}}, & \text{otherwise}, \end{cases}$ (5)

where L is the set of statute and Constitution chunks, w_{court} is 1.0 for the Supreme Court, 0.7 for a High Court, 0.4 for a tribunal and 0.3 when unknown w_{rec} is 1.0, 0.8, 0.6 or 0.4 for decisions at most 2, 5, 10 or 20 years old and 0.2 otherwise or when undated and $w_{\mathrm{cit}} = \min(n_{\mathrm{cit}}/100, 1)$ with a default of 0.5 when the citation count n_{cit} is unknown. The RRF score is rescaled to [0,1] by the rank one value of a single list, $\tilde{R}(c) = \min(R(c)\,(k+1),\ 1)$, and the final ranking score is

$$S(c) = w_s\, s(q,c) + w_r\, \tilde{R}(c) + w_a\, A(c),$$

with $(w_s, w_r, w_a) = (0.50, 0.25, 0.25)$. Exact match hits that the dense path did not return are scored against the query embedding as well, so they compete on equal terms. The top $K = 4$ chunks by S are passed to the generator.

H. Grounded Generation

The prompt numbers the retrieved passages as `[SOURCE_1]` … `[SOURCE_n]`, each of them with its own document type and metadata (case name, court, Act and section), and appends the IPC/BNS table as the last source when it matched. The system also prompt requires the model to (i) use only the sources, (ii) put a citation after every factual statement, (iii) never invent case names, reporter citations, section numbers or quotations, (iv) state both of the IPC and BNS sections when the table is present now , (v) explain which source carries more authority when sources disagree, and (vi) answer the covered part of a partially answerable question and then name what is missing, refusing only when them no source is relevant. Generation also uses an OpenAI compatible client against NVIDIA NIM with temperature 0.1 and at most 1,024 output tokens, with retries and a client side rate limit of 40 requests per minute. Any OpenAI compatible model can be configured DeepSeek and NVIDIA Nemotron were used in the recorded runs.

I. Citation Validation and Confidence Scoring

A parser extracts every `[SOURCE_N]` tag, maps it to the passage that was sent and flags tags that point to no passage. The confidence score combines four signals

$$C = 0.40\, Q + 0.25\, G + 0.20\, H + 0.15\, V,$$

where $Q = \mathrm{clip}((\bar{S} - 0.3)/0.6,\ 0,\ 1)$ maps the mean final score $\bar{S}$ of the retrieved chunks to [0,1] G is source agreement, equal to 0, 0.3, 0.6 or 1.0 when 0, 1, 2 or at least 3 distinct sources are cited H is the mean authority of the cited chunks and V is the fraction of retrieved chunks that were cited. $C \geq 0.8$ is reported as High, $C \geq 0.6$ as Medium, $C \geq 0.4$ as Low and lower values as Very Low.

J. Training and Inference

Nyaya has no trainable parameters the embedding model and the LLM are used frozen in our project, so there is no loss function, optimiser or training loop, and only the retrieval and fusion parameters are chosen from the data. At inference the stages of Fig 1 run in order and return the answer, all numbered sources, invalid citations, IPC and BNS matches and the confiidence breakdown, through a command line tool or a Python API.

IV. Experimental Results

The experiments were run on an Apple M2 laptop, with Python 3.12,PyTorch 2.14,Sentence Transformers 6.2, ChromaDB 1.5 and PyMuPDF. Embedding and retrieval runned locally on the CPU generation runs remotely on NVIDIA NIM. Building the index (download, conversion, chunking and embedding of 112,555 chunks) takes about 15 minutes, of which embedding takes about 11.

Because the system is not trained, there is no train/test split and no epochs, batch size or learning rate. The full corpus is the retrieval index and all 50 queries are used for testing. No separate validation are set exists, so the parameters in Section IV D were tuned on the same queries and those results should be read as a sensitivity analysis now and not as an unbiased estimates for new queries. Unless stated otherwise, experiments use a pool of 20 per path, $K = 4$, $k = 60$, temperature 0.1 and 1,024 output tokens.

A. Dataset Description

Table I lists all the five dataset. Record that arrive before splitting are index as they are, all the other records pass with a structure awared chunkers . Thus resulting in 101,983 chunked and 12,041 pre split passages giving 112,555 unique chunks after duplication identifiers are merged. Then the corpus is highly imbalanced judgments can make up to 76.4% of the total chunks, Constitution passages the 19.9% and other passages only just 3.7%, and default build does not contain just text of every IPC and BNS section. Many jugments chunks also lack cases name, date or total citation count, thus their authority score falls back to the defaults. All this data is

publicly released legal material, it contains all the names of the rule already have been published in the court records but in it there was no other personal data that was collected.

TABLE I. Knowledge base sources (Hugging Face).

Dataset	Records	Content
Viber1/indian law dataset	24,607	Statute Q&A pairs
Ninadn/indian legal	7,130	SC judgments, summaries
Vihaannnn/Indian SC Chunked	20,961	Pre split SC judgments
ShreyasP123/Legal Dataset	1,469	Pre split legal texts
nisaar/Constitution_of_India	933	Constitution articles
Indexed chunks are 112,555 (judgment 87,099 constitution 22,654 statute 4,271 before merging duplicate identifiers)		

In this none of public benchmark cover all the cited question that are answering over Indian rules, the Constitution and the BNS(Bharatiya Nyaya Sanhita), thus a test set of just 50 queries was built by bare hands (Table II).In this each query has only a category, that the expected document types and a list of expected keywords gives. All the sectional lookup ask for a particular provisions ("Define Section 498A of IPC"), case lookups or just ask what a name jugment held ("What was held in Maneka Gandhi v Union of India?"), conceptual and theoretical queries ask just about a doctrines ("Explain the doctrine of basic structure"), and referenced queries compare IPC and BNS or CrPC and BNSS, and all the edge cases tests robustness with a empty string .

TABLE II. Composition of the 50 query test set.

Category	Queries	Example
Sectional lookup	15	What is Section 302 IPC?
Case lookup	10	Summarize Navtej Singh Johar v UoI
Conceptual	10	Explain the basic structure doctrine
Cross reference	10	Compare IPC and BNS on murder
Edge case	5	Section 99999 of IPC empty query

B. Evaluation Metrics

All of the metrics are predictable and thus computed by a evaluation script without a LLM judge. For a query with ge set D_K of K chunks, expected keywords $\mathcal{W}$ and expected types $\mathcal{T}$

Retrieval precision is a share of all of the retrieved chunks that are of a predicted type or contain an expected keyword,

$$P@K = \frac{1}{K}\sum_{c\in D_K} \mathbf{1}\,[\text{type}(c) \in \mathcal{T} \vee \exists w \in \mathcal{W}: w \in c]. \qquad (8)$$

because of the type thing alone is easy to satisfy, it also reports the stricter *keyword precision* $P_{\text{kw}}@K$, which counts only the chunk containing an expected keyword *Hit@K*, and the share of each of the query with at least one keyword releated chunk, and a *mean reciprocal rank*

$\text{MRR} = \frac{1}{|Q|}\sum_q 1\,/\text{rank}_q$ of the first keyword relevant chunk.

Citation validity is just a part of `[SOURCE_N]` tags in a answer with $1 \le N \le n$, where n is all the numbers of source sent, and a answer with no tag scores 0. *Cited claim coverage* is just a fraction of all legal claim sentences (sentences of at least 25 characters that mention a legal thing such as section, Act, court or offence) that carry a citation. *Citation support* is a fraction of all of the cited claim whose sources contain same section for article numbers and legal concept and shares minimum 35% of claim's content words or at least 5 among them. *Hallucination rate* is just a fraction of all answers that name a case ("X v Y") or a AIR/SCC/SCR citation found in none of the lsited source in the list. *Response rate* is a fraction of all answers that cite at least one of the valid source and are just not at all a refusal. *Average confidence* is total mean of C.

C. Comparison with Baseline Methods

Table III compares all of proposed retriever with the three baselines on a 49 non empty test queries,which all use same indexes,the embedding models, IPC/BNS expansions, de duplications and $K = 4$ i.e dense only retrieval as in a standard RAG [1], [7], which matches exact retrieval only, and also hybrid retrieval fused with a standard fixed type of weight RRF [2]. Rows (e) and (f) isolate all the contribution of queries adaptive weights and a authority prior.

TABLE III. Retrieval comparison with the baseline methods (49 queries,).

Method	P@4	P_{kw}@4	Hit@4	MRR
(a) Dense only (standard RAG)	0.867	0.656	0.796	0.719
(b) Exact match only	0.714	0.694	0.694	0.694
(c) Hybrid, fixed RRF (0.5/0.5)	0.913	0.776	0.878	0.837
(d) Hybrid, adaptive RRF	**0.939**	**0.842**	**0.878**	**0.878**
(e) (d) + re rank, no authority	**0.939**	**0.842**	**0.878**	**0.878**
(f) Proposed (e) + authority	0.934	0.811	**0.878**	**0.878**

Hybrid retrieval is thus better than either of the paths alone. Dense retrieval does well on conceptual and cross reference questions (keyword precision 0.925 and 0.975) but it usually collapses on a case lookups (0.225), because a embedding of "Maneka Gandhi v Union of India" is closer to any of the passage about the personal liberty or personal freedom or any such thing than to judgment itself. More precised matches retrieval has the opposite profile 1.000 on sectional lookups but 0.100 on all of the conceptual queries, though which genrally contains none of the identifiers. Fusion keeps the strengths of both of them , and thus making all of the weights depended on the queries type that adds 6.6 point of keywords precision and the 4.1 points of a MRR over the fixed RRF, almost entirely on the particular cases lookups (0.475 to 0.600) and sectional lookups (0.883 to 1.000).Thus removing the duplicated passages do not change the precision but lifts up the Hit@4 of dense retrieval from the 0.776 to 0.796, because the 19.4% of total densed top 4 slots were though other than that copies of other passages which were already shown.

However the authority prior costs only 3.1 points of the total number of keyword precisions (row f against row e) and do not changes the Hit@4 or MRR. It defines rules and Constitution passages, which genrally scores 1.0, above judgments whose metadata are usually missing and cannot be found and which thus receives defaulted authority values for the case links and theoretical questions this sometimes replaces the relevant judgments paragraphs with the lesser particular constitutional passages. Then the prior ones is kept in a deployed system because generally it orders a conflicting sources by the binding forces, which the keyword based metric cannot usually get, but though this result shows that it

should be re tuned once a judgments metadata is more complete than that it looks.

The three of the recorded every end runs are compared in the Table IV whcih we can observe. Run A used the DeepSeek V3.2, 6 passages and an earlier prompt that asked model to be refuse whenever coverage was not completed, it recorded just citation validity under a name "citation accuracy". Runs B and C uses current prompts and scoring fixes with the `nemotron 3 ultra 550b a55b,` and the Run C adds cases names and the much more nearest of the matches exact search. Run A's high validity gives mostly the refusals in which 35 of its all 50 answers were not good. Thus the current prompt was more than the 2 times the response rate (0.30 to 0.68) while genrally keeping all the hallucinations at one or two answers in fifty with it. Validity usually falls in it because it is averaged over all of the answers and an answer without any tag scores 0, so it tracks responses rates in that with this.

TABLE IV. End to end results on a 50 query tests set.

Metric	Run A (DeepSeek, old)	Run B (Nemotron)	Run C (Nemotron, final)
Citation validity	0.94	0.66	**0.70**
Cited claim coverage		0.42	0.42
Citation support		0.50	0.50
Hallucination rate ↓	**0.02**	**0.02**	0.04
Response rate	0.30	0.66	**0.68**
Retrieval precision	0.79	0.91	**0.92**
Average confidence	**0.69**	0.57	0.61
Mean latency (s)	25.0	15.4	17.5

Table V breaks down a Run C by the categories. Sectional lookups and theoretical questions are all answered extremely well, with response rates of the 0.93 and 0.80 and thus giving citation supports of 0.64 and 0.80. In this case lookups are weakest category retrieval precision and accuracy is a perfect under the lighter metrics, but corpus often usually holds only tye fragments of a named judgment without its case name, and the model then declines it to an attribute . Cross references question suffers from the missing source texts, the BNSS and provisions of BNS on an organised crime and seditions are not in default corpuses, and thus model correctly reports that the sources usally do not cover up them. Thus the edge cases behave as an intended empty queries are rejected, and non existent "Section 99999" produces a refusal instead of an invented provisions. Manual inspections of 2 hallucinations flag one is model writing out its reasoning ("The user is asking what was held in …"), the other one is the single phrases like "Federal Court vs Privy Council" in the history of all the law.

TABLE V. Per categories results of Run C.

Category	Validity	Support	Resp.	Ret. P	Conf.
Section lookup (15)	0.93	0.64	0.93	0.98	0.78
Case lookup (10)	0.50	0.32	0.50	1.00	0.46
Conceptual (10)	0.90	0.80	0.80	1.00	0.74
Cross reference (10)	0.60	0.35	0.60	1.00	0.49
Edge case (5)	0.20	0.10	0.20	0.20	0.35

Ussually Commercial legal tools cannot be compared using numbers , since Magesh *et al.* [3] evaluated them on US law with expert thinking. There 17% to 33% hallucinations rates are above the mark of the 4% flagged here, but our detector cant identify all of the wrong statements attributed to the real sources, so true error rate is higher than the citation support score of 0.50 is one of the more cautious indicators than others

D. Hyperparameter Tuning

With all of the usual frozen models,all the tunable quantities are one of the numbers of passages K, the candidates pools per path of them, the RRF's constants k,and other score weights (w_s, w_r, w_a) and also the per classes fusion weights in it. Each one in it varies one at a time around unsual deployed configuration by the grid searches on the cached search results in it for the 49 non empty ones queries, such that every of the configurations ses identicals or exactly similar candidates in that. keywords precision and Hit@4 are reported because the lighter ones $P@4$ barely or rarely changes in that.

TABLE VI. Effect of the number of all the passages K given to the LLM.

K	$P@K$	$P_{\text{kw}}@K$	Hit@K	MRR
2	0.939	**0.867**	0.878	0.878
4	0.934	0.811	0.878	0.878
6	0.932	0.776	**0.898**	**0.882**
8	0.929	0.758	**0.898**	**0.882**
10	0.931	0.754	**0.898**	**0.882**

Table VI shows all of the expected precision recalls trade offs . The smallers K gives much more purer contexts but leaves the models with fewer and much more lower distinct sources give, and the sources agreement signal of one of the confidence score required 3 cited sources for the all credits. Thus, Increasing K beyond 6 adds no other further hits and only compresses the contexts, which also places the relevant pragraphes in the middle of a much more longer prompt where LLMs use them less consistenly [12]. $K = 4$ was selected as the smallest value among it that allows multi sources answers while keeping all the keywords precisions above 0.8 always. Run A used $K = 6$, compressing it to a 4 also shorten the prompt, which contributes to the lower latency of Runs B and C, though the model also changed but still it gets that.

TABLE VII. Effect of the RRF constant k and candidate pool size.

RRF k	$P_{\text{kw}}@4$	Hit@4	Pool	$P_{\text{kw}}@4$	MRR
1	0.750	0.878	5	**0.815**	**0.878**
10	0.791	0.878	10	0.801	**0.878**
30	0.801	0.878	**20**	0.811	**0.878**
60	0.811	0.878	30	0.806	0.867
100	**0.816**	0.878	40	0.806	0.867

Table VII gives us that very small RRF constants hurt with$k = 1$ rank one item of each of the lists dominates the and keywords precisions drop to 0.750. Which is in between $k = 30$ and $k = 100$ results are within 1.5 points. The candidate ones usually matters a little between the 5 and 40, but pool above 20 admits more and more thin candidates and lower MRR slightly, so 20 was kept beacuse of that.

TABLE VIII. Effect of final score weights (w_s, w_r, w_a) and of the fixed versus adaptive fusion weights.

(w_s, w_r, w_a)	P_{kw}	Hit	Fusion (α, β)	P_{kw}	Hit
(1.0, 0, 0)	0.668	0.837	fixed (1.0, 0.0)	0.597	0.755
(0.7, 0.3, 0)	0.827	0.878	fixed (0.8, 0.2)	0.643	0.796
(0.5, 0.5, 0)	**0.842**	0.878	fixed (0.5, 0.5)	0.719	0.837
(0.6, 0.2, 0.2)	0.786	0.878	fixed (0.2, 0.8)	0.821	0.878
(0.4, 0.4, 0.2)	0.827	0.878	fixed (0.0, 1.0)	0.827	0.878

(w_s, w_r, w_a)	P_{kw}	Hit	Fusion (α, β)	P_{kw}	Hit
(0.34, 0.33, 0.33)	0.827	0.878			
(0.5, 0.25, 0.25)	0.811	0.878	**adaptive**	0.811	0.878

Table VIII (left) thus confirms that a reranker using semantic similarities alone loses all of the benefits of fusion (0.668), and that any substantial RRF weight recovers it usually. The one of best keyword precision, 0.842, is obtained, the deployed weights (0.5, 0.25, 0.25) trade 3.1 points for a upper preference of binding sources, a design choice rather than a metric. On the right side of it, weighing the exactly same match path more gradually helps becauseof the most of test queries contains the identifiers, it presets matches the best of the fixed setting while keeping all a high semantic weight for conceptual and theoretcial queries, which however have no exact same matched results like that.

E. Local Answer Writer: Nyaya-Gen

All of the Runs A to C that are above depends on the API model for writing of the answer, so every query and its all retrieved passages leaves the laptop, it needs a network and an API key, and each of the request has a cost in it. The system described in Section III has no trainable parameters, thus as a last step we trained only one component, a small answer writer which is called Nyaya-Gen, that replaces the API model while the retrieval, prompt, citation validation and confidence scoring are kept exactly same. Nothing else in the pipeline is changed, so any difference between the runs usually comes from the writer only.

1) Model and objective: We used the SmolLM2-135M-Instruct [15], a decoder only model with 134.5M parameters which is about 4,000 times much more smaller than the 550B API model, and trained all of its weights on our task. Each training example is the exact same prompt x that the pipeline sends to the LLM (system rules, numbered sources and the question) and a target answer y that cites the right source as [SOURCE_k]. The loss is the next token cross entropy computed only over the answer tokens A, so model learns to write the answer and does not learn to repeat the prompt,

$$J_G(\theta) = -\frac{1}{|A|} \sum_{t \in A} \log p_\theta(y_t \mid y_{<t}, x) \qquad (9)$$

where θ are all of the model weights and the prompt tokens are masked out of the loss.

2) Training data: No labelled legal answers were available, so the data was built from the same public Q&A datasets that are indexed (statute and Constitution question answer records). For a Q&A example the own record of the question is hidden at a random position k among 2 to 4 distractor passages that are taken randomly from whole corpus, and the target restates the answer of that record in at most four sentences with each sentence cited as [SOURCE_k]. Mapping examples ask for the BNS equivalent of an IPC section and use the IPC/BNS table chunk as a source, and refusal examples leave out the own record, so the target is the refusal sentence of the system prompt. To keep the test set unseen, 572 questions which have a word Jaccard overlap of 0.5 or more with any of the 50 test queries were removed, and the mapping rows of IPC sections that are in test queries were skipped. The records are split by record into train, dev and test (94%, 3% and 3%), so no record appears in two of the splits. Each passage is cut to 90 words so that examples fits in the 8 GB memory, and the examples longer than 1,024 tokens are dropped. Table IX gives all of the counts.

TABLE IX. Training data of the Nyaya-Gen.

Kind	Examples	What the target is
Q&A, cited	3,590	answer of own record, each sentence cited [SOURCE_k]
IPC/BNS mapping	141	BNS equivalent, cited to the table chunk
Refusal	269	refusal sentence (no relevant source)
Total built	4,000	cap on training set
Used (≤ 1,024 tokens)	2,585	median length 931 tokens
Dev (checkpoint choice)	24	dev loss only
Removed by leakage filter	572	overlap with the test queries

3) Training: Training is done on the same MacBook Air M2 with 8 GB memory, on the MPS GPU back end in fp32, with the gradient checkpointing [17] for fitting it in the memory; Table X lists the settings. AdamW [16] with the learning rate 1×10^{-4} is used, with 20 warm up updates and then a cosine decay, and a gradient accumulation of 8 so each of the update sees 8 examples. One epoch would be 323 updates, but each update took about 0.8 minute and after about 80 updates in 65 minutes the laptop ran out of memory (the swap was full) and training stalled, so it was stopped. The checkpoint with the lowest dev loss, at update 50 (about 400 examples, 15% of one epoch), is kept as Nyaya-Gen. The dev loss falls from 0.833 before the training to 0.033 at that update and the training loss from 0.98 to about 0.01 (Fig. 2), because the format of answer is fixed and it is learned very fast. A low loss does not prove that answers are correct, thus the end to end test below is the real test of it.

TABLE X. Training settings of the Nyaya-Gen.

Setting	Value
Base model	SmolLM2-135M-Instruct [15], 134.5M parameters
Hardware	Apple M2, 8 GB, MPS back end, fp32
Memory saving	gradient checkpointing [17], 90 word passages
Optimiser	AdamW [16], lr 1×10^{-4}, β = (0.9, 0.95)
Schedule	20 warm up updates, then cosine decay
Batch	1 example × 8 accumulation = 8 per update
Max length	1,024 tokens
Planned / done	323 updates (1 epoch) / about 80 in 65 min
Kept checkpoint	update 50 (lowest dev loss)
Dev loss	0.833 → 0.033
Decoding	greedy, repetition penalty 1.1, ≤ 256 new tokens

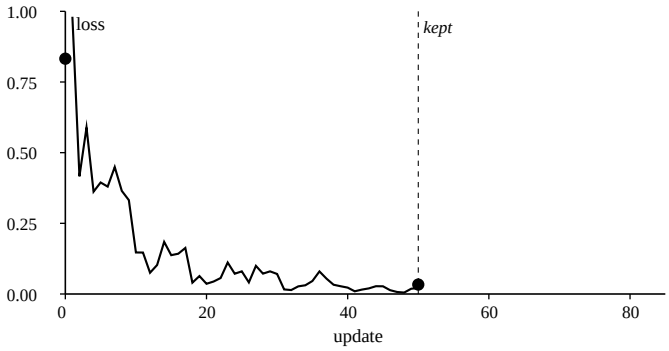


Fig. 2. Training loss (line) and dev loss (dots) of Nyaya-Gen for each update. The run stopped at about update 80; the checkpoint at update 50 is kept.

4) End to end results: Table XI compares Run C with Run D, in which Nyaya-Gen writes the answer fully on the laptop for the same 50 test queries, with the same retrieval, prompt and the scoring. Nyaya-Gen gets a citation validity of 0.74 against 0.70, a cited claim coverage of 0.66 against 0.42, a citation support of 0.56 against 0.50 and a response rate of 0.72 against 0.68, and the hallucination detector flags none of its answers (0.00 against 0.04). Retrieval precision is the same 0.92 because retrieval is not changed. The average confidence is lower (0.45 against 0.61) since Nyaya-Gen usually cites only one source in an answer, so the source agreement part of the confidence score stays low.

TABLE XI. End to end results of the API writer (Run C) and the Nyaya-Gen (Run D) on a 50 query tests set.

Metric	Run C (Nemotron, API)	Run D (Nyaya-Gen, local)
Citation validity	0.70	**0.74**
Cited claim coverage	0.42	**0.66**
Citation support	0.50	**0.56**
Hallucination rate ↓	0.04	**0.00**
Response rate	0.68	**0.72**
Retrieval precision	0.92	0.92
Average confidence	**0.61**	0.45
Mean latency (s)	**17.5**	45.0
Writer parameters	about 550B	134.5M
Network needed	yes	no

The mean latency is 45.0 s against 17.5 s, but most of it is the exact match scan, which takes about 62 s on the queries that have an identifier when the generator is also loaded and 8 GB memory swaps, and the writing of the answer itself takes only about 11 to 15 s. Run D needs no network, no API key and it has no cost per request.

5) Standard metrics: To compare with the usual way an LLM is evaluated in the industry, both the Nyaya-Gen and the same SmolLM2-135M before our training answered the 200 held out test examples of its data (181 cited answers and 19 refusals, never seen in the training), and their answers are compared with the gold answer by three common metrics with the [SOURCE_*k*] tags removed: token F1 (the word overlap used in the SQuAD), ROUGE-L F1 (the longest common word sequence) and BERTScore F1 (meaning similarity from BERT embeddings, here with DistilBERT). Table XII shows that our training raises the token F1 from 0.42 to 0.93, the ROUGE-L from 0.40 to 0.93 and the BERTScore from 0.83 to 0.98. These scores are high because the target usually restates the right source closely, so they show mostly that the model learned to find and restate the right source in the required format and not a deeper legal reasoning.

TABLE XII. Standard LLM metrics on the 200 held out test examples of the Nyaya-Gen data.

Metric	SmolLM2-135M (before training)	Nyaya-Gen
Token F1	0.42	**0.93**
ROUGE-L F1	0.40	**0.93**
BERTScore F1	0.83	**0.98**

These results should be read much more carefully. Answers of the Nyaya-Gen are much shorter and plainer, because its training targets restate one source in at most four sentences, so it was never taught to combine the several sources or to compare them by their authority, and the six metrics do not measure these qualities. It was trained for only 50 updates with one seed, and the same small model before our training was compared only on the held out examples of Table XII and not on the 50 end to end test queries.

V. Conclusion

Thus this project addresses the unreliability of all of the LLMs as the sources of all Indian laws by building the Nyaya, RAG system in which that model usually only writes from retrieved, and numbered of the passages. Though structure aware chunking of the 5 available public datasets produced the 112,555 chunked indexes i.e query adaptives hybrid retrieval with weighted RRF raises keywords grounded precision at upto 4 passages from the 0.656 for the much more denser retrieval to 0.811 and Hit@4 from 0.796 to 0.878, and an IPC/BNS table done as a citable source lets our system answer questions. Also with the citation validation and much more revised prompt, the system answers 68% of all test queries with valid citations, and with only of two answers out of the 50 flagged by hallucination detector.Howver, limitations remain like the 50 query tests set was also used for tuning, MiniLM was'nt compared with other legal encoders ones such as the one InLegalBERT, the defaulter group lacks the full BNS and BNSS, the IPC/BNS table covers 63 sections, the exact same match scan takes the seconds per query, and the hallucinations detector missesall the incorrect statements attributed to the authentic sources.

In addition, a 135M parameter answer writer, Nyaya-Gen, that was trained on the same 8 GB laptop in about one hour lets the whole system run offline, and on the 50 test queries it reached a citation validity of 0.74 and a response rate of 0.72 with no flagged hallucinations, against 0.70, 0.68 and 0.04 for the 550B API model. Its training was cut short by the memory of the laptop, so the longer training, answers which combine several sources and a comparison with the model before

training are left for the future work.

REFERENCES

[1] P. Lewis E. Perez A. Piktus F. Petroni V. Karpukhin N. Goyal H. Küttler M. Lewis W. Yih T. Rocktäschel S. Riedel and D. Kiela "Retrieval augmented generation for the knowledge intensives NLP tasks," in *Advances in Neural Information Processing Systems* .

[2] G. V. Cormack C. L. A. Clarke and S. Büttcher, "Reciprocal rank fusion outperform the Condorcet and individual ranks learning methods " in *Proc. 32nd Int. ACM SIGIR Conf. Research and Development in Information Retrieval* .

[3] V. Magesh F. Surani M. Dahl M. Suzgun C. D. Manning and D. E. Ho "Hallucination free? Assessing the reliability of leading AI legal research tools," *Journal of Empirical Legal Studies.*

[4] M. Dahl V. Magesh M. Suzgun and D. E. Ho "Large legal fictions Profiling the legal hallucinations in large language models," *Journal of Legal Analysis*.

[5] S. Paul A. Mandal P. Goyal and S. Ghosh "Pre trained language models for legal field A case study on Indian law," in *Proc. 19th Int. Conf. Artificial Intelligence and Law* .

[6] I. Chalkidis M. Fergadiotis P. Malakasiotis N. Aletras and I. Androutsopoulos, "LEGAL BERT straight out of law school," in *Findings of the Association for Computational Linguistics* .

[7] V. Karpukhin B. Oğuz S. Min P. Lewis L. Wu S. Edunov D. Chen and W. Yih "Dense passage retrieval for open field question answering," in *Proc. Conf. Empirical Methods in Natural Language Processing* .

[8] N. Reimers and I. Gurevych "Sentence BERT Sentence embeddings using Siamese BERT networks," in *Proc. Conf. Empirical Methods in Natural Language Processing and 9th Int. Joint Conf. Natural Language Processing* .

[9] W. Wang F. Wei L. Dong H. Bao N. Yang and M. Zhou "MiniLM Deep self attention distillation for task agnostic compression of pre trained transformers," in *Advances in Neural Information Processing Systems*.

[10] S. Robertson and H. Zaragoza, "The probabilistic relevance framework BM25 and beyond," *Foundations and Trends in Information Retrieval.*

[11] S. Es, J. James L. Espinosa Anke and S. Schockaert "RAGAS Automated evaluation of retrieval augmented generation," in *Proc. 18th Conf. European Chapter of the Association for Computational Linguistics System Demonstrations*, 2024.

[12] N. Guha *et al.*, "LegalBench A collaboratively built benchmark for measuring legal reasoning in large language models," in *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*.

[13] DeepSeek AI, "DeepSeek V3 technical report," arXiv preprint arX.

[14] Government of India *The Bharatiya Nyaya Sanhita, 2023 (Act No. 45 of 2023)*. New Delhi Ministry of Law and Justice, Gazette of India, 2023.

[15] L. B. Allal A. Lozhkov E. Bakouch G. M. Blázquez G. Penedo L. Tunstall *et al.*, "SmolLM2 When smol goes big, data centric training of a small language model," arXiv preprint arXiv:2502.02737, 2025.

[16] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. Int. Conf. Learning Representations*, 2019.

[17] T. Chen B. Xu C. Zhang and C. Guestrin, "Training deep nets with sublinear memory cost," arXiv preprint arXiv:1604.06174, 2016.

[18] A. Paszke S. Gross F. Massa *et al.*, "PyTorch An imperative style, high performance deep learning library," in *Advances in Neural Information Processing Systems*, 2019.